

Foundations for programming languages

10: Object oriented languages

Isao Sasano

Division of programs

It is better to develop large programs by dividing them into smaller programs.

Procedures provide one of the most basic ways for division.

Modules provide a way to put together variables, procedures, and types which relate to each other. Modula-2 (a successor of Pascal) supports modules as its construct.

Modules enable us to hide implementation of data types processed by procedures.

Modules

Modules group declarations of variables, procedures, types, and so on. By exporting types in modules, we effectively obtain user-defined data types.

[Implementation of stack in a module (in Modula-2 like notation)]

```
type stack;  
function pop(s: stack): integer;  
procedure push(a: integer; s: stack);  
function newstack: stack;
```

Interface

```
type stack = ↑rep;  
      rep = record ...  
function newstack: stack;  
  var s: stack;  
begin ...
```

Implementation

Abstract data types

Abstract data types consist of types and operations on them. They are new types similarly to the built-in types such as `int`.

`Int` type accompanies operations `+`, `-`, `*`, etc. If the language is defined so that accessing to the values of built-in types is only allowed through the accompanying operations, the behaviors of programs are not affected even if the representations of the built-in types are different in different computer architectures. Similarly, behaviors of programs with abstract data types are not affected even if their implementation are changed.

Abstract data types (cont.)

Standard ML supports abstract data types. Standard ML is statically typed and checks whether or not the values of abstract data types are only accessed by the accompanying operations.

(Programs that access the values of abstract data types not by the accompanying operations become type error in compile time.)

Abstract data types (cont.)

A Standard ML program defining an abstract data type for complex numbers:

```
abstype complex = C of int * int with
  fun complex (x,y: int) = C (x,y)
  fun x_coord (C(x,y)) = x
  fun y_coord (C(x,y)) = y
  fun add (C(x1,y1), C(x2,y2)) = C(x1+x2, y1+y2)
end
```

We can change the definition of the four functions without affecting the programs using this type.

A program containing this definition is put on my lecture web page.

Abstract data types (cont.)

- Comparison between abstract data types and modules
 - An abstract data type gives a type, while a module may group more than one data types (which may relate to each other).

Classes in C++

Classes in C++ are generalizations of records, which are called structures in C.

After declaring a class, we can use it as a type name, declare variables of the type, and generate an object of the type.

(ex.)

```
struct Stack {  
    int top;  
    char elements [101];  
    char pop();  
    void push (char);  
        Stack();  
};
```

```
class Stack {  
    public:  
        int top;  
        char elements [101];  
        char pop();  
        void push (char);  
            Stack();  
};
```

(Cf.) C++ is designed by porting classes in Simula 67 to C.

Class declarations in C++

The declaration

```
struct X { <declarations> };
```

is equivalent to the declaration

```
class X { public : <declarations> }
```

and the declaration

```
class X { <declarations> };
```

is equivalent to the declaration

```
struct X { private : <declarations> };
```

An example of class declaration

```
class Stack {  
    int top;  
    int size;  
    char *elements;  
public:  
    Stack (int n) {size=n;  
                  elements = new char[size]; top=0;}  
    ~Stack()      { delete elements; }  
    void push (char a) {top++; elements[top] = a;}  
    char pop()      {top--; return elements[top+1];}  
};
```

Generation and deletion of objects

In C++ objects are generated by **new** on the heap and deleted by **delete**. For any type T , the expression

new T

generates an object of type T and the value of the expression is the pointer to the generated object.

delete p

deletes the object pointed by p .

Objects can be generated on the stack as well by local variable declarations. In this case, objects are deleted automatically when the control reaches at the end of the block where the variables are declared.

See the programs on the lecture web page.

C++ templates: an example

```
template <class T> class Stack {  
    int top;  
    int size;  
    T * elements;  
public:  
    Stack (int n) {size=n; elements = new T[size]; top=0;}  
    ~Stack()      { delete elements; }  
    void push (T a) { top++; elements[top]=a; }  
    T pop()        { top--; return elements[top+1]; }  
};
```

T is replaced with some type like Stack <int>. The expression **new** Stack<int> (99) generates a Stack object whose elements are of type int.

C++ template is compiled to fast code, whereas code is duplicated for each type instantiation.

Object-oriented languages

Object-oriented languages were invented to describe some simulations. Objects are elements in simulations.

(ex.) Simula (Simulation language, 1967)

- It was developed by Ole-Johan Dahl, Kristen Nygaard.
- It is the first object-oriented language, although the term “object-oriented” were not used at the time.
- It was designed as an extension of ALGOL.
- Description of airport system was an important example.

Object-oriented languages

Examples of objects in airports: customers, counters, queues, tickets, etc.

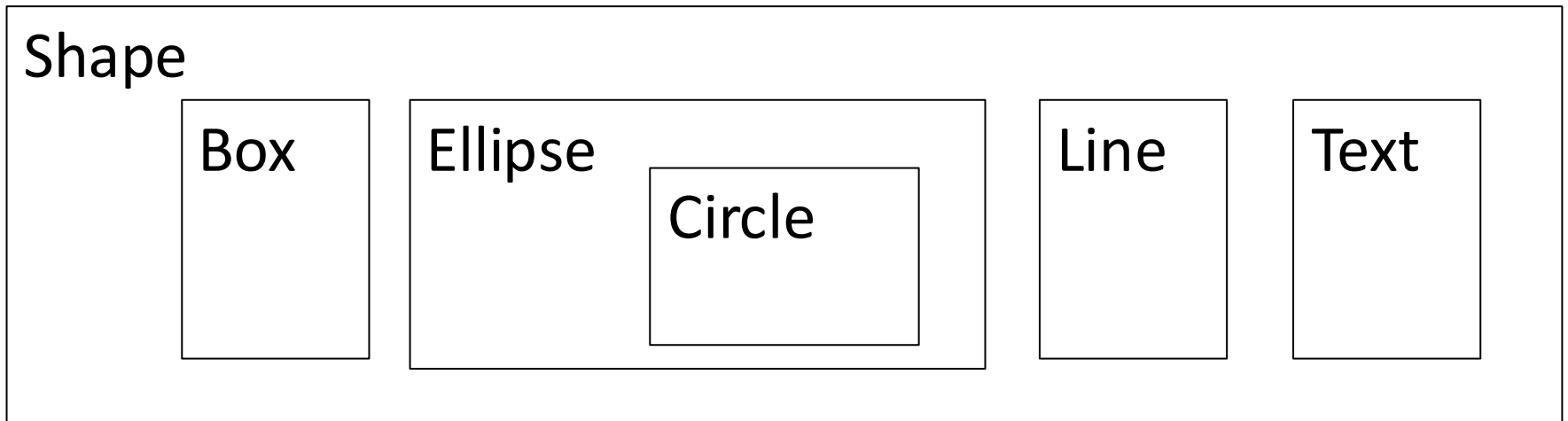
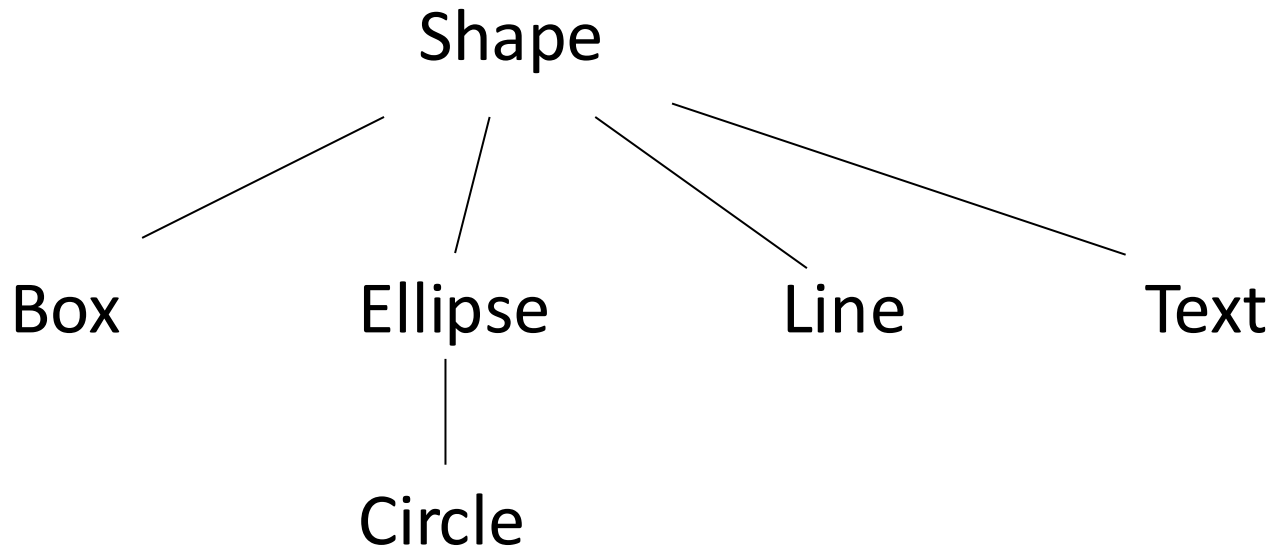
(external view)

Computations proceed by passing messages between objects.

(internal view)

When receiving a message, a corresponding function is executed. The function is referred to as a *method* or a *member function*.

Hierarchy of classes



Inheritance

- A derived class inherits the member variables and member functions of its parent (base) class. (The member variables and member functions of the base class becomes the ones in its derived classes.)
- In the derived classes, member variables and member functions can be additionally defined. When the name is the same, the definition is overridden.

(Note) “overload” is different from “override”. “Overload” is to define methods of the same name with the the number of arguments or the types of arguments being different.

Overloading

- You can download a C++ program using overloading from my lecture web page.

Examples in C++

In C++, inheritance is described as follows.

```
class Box : public Shape {  
    ...  
}
```

Inherit all the members with keeping the visibility.

```
class Box : private Shape {  
    ...  
}
```

Inherited members are private by default.

In C++, super classes (or parent classes) are called *base classes* and subclasses (or child classes) are called *derived classes*.

Virtual functions

By Attaching the keyword “virtual” to a method declaration, which of the methods is executed is determined in runtime, not in compile time.

A C++ program for showing a typical usage of virtual functions is put on my lecture web page.

Features of C++

- C++ was designed to keep the backward compatibility with C as much as possible.
- Objects are extension of structures in C. That is, objects can be allocated on the activation records (in the stack) as well as on the heap.
- Imperative style of programming is possible. C++ does not force the programmer to program in object-oriented style.
- Multiple inheritance is supported. (This is out of the scope of this class.)

Features of OO language

OO languages have the following features.

- *Dynamic lookup* (which of the methods is executed is determined in runtime.)
- *Abstraction* (public members serve as interface and private ones are hidden from the outside.)
- *Subtyping* (Where objects of some class are expected, objects of the classes that inherits the class with the public keyword can be used.)
- *Inheritance* (Some code can be reused without copy-and-pasting.)

Subtyping

- *Subtyping* is supported in some type systems.
- A reference:
 - Atsushi Igarashi, Benjamin Pierce, Philip Wadler, “Featherweight Java: a minimal core calculus for Java and GJ”, ACM TOPLAS 23(3):396-450, 2001.
 - You can download freely at <https://dl.acm.org/doi/10.1145/503502.503505> from inside this institute.

Note

Subtyping and inheritance are different.

(ex.)

Queue --- first-in, first-out

Stack --- last-in, first-out

Deque --- queue where inputs and outputs can be done in the both side

Queue class and Stack class can be implemented as a derived class of Dequeue class by privately inheriting members by default and make public some methods.

But Queue and Stack are not subtype of Dequeue.

An example

- A C++ program for showing that subtyping and inheritance are different is put on my lecture web page.